

NanoPoreSeq

NanoPoreSeq — User guide

Simulate Nanopore BAMs of brain tumours and test Rapid-CNS2_nf

Version of 26 October 2026 (2) · scripts of the simulation/, rapidcns2/ and usb/ folders

This project is developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.

Contents

1. About this guide
2. Overview
3. Quick start
4. Installation
5. Key concepts
6. How to
7. Command reference
8. File reference
9. Resources and run times
10. Troubleshooting
11. Glossary
12. Appendix: code for generating the USB stick

1. About this guide

This guide explains how to install and use NanoPoreSeq: simulate Nanopore BAMs of gliomas, check them, obtain their methylation class, simulate cohorts, then use them to test the Rapid-CNS2_nf pipeline, including from a self-contained USB stick.

It is meant for bioinformaticians and Rapid-CNS2 users comfortable with the Linux command line. The internal workings and the history of the corrections are described separately in the review report (docs/ folder).

Development

This project is developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady. The code, tests and documentation are produced with the assistant, then reviewed, discussed and validated by the author; the tests reported in this guide were actually run.

Conventions

- Commands are typed in a terminal, from the folder shown (simulation/, rapidcns2/ or usb/).
- $DATA stands for the data/ folder of Rapid-CNS2_nf and $REF for the hg38 FASTA (see 4.2). A "\" at the end of a line means the command continues on the next line.
- Names between angle brackets, such as <id>, are to be replaced with the actual value (sample name, folder…).
- The run times and memory figures quoted were measured on a test machine with 7 GB of memory, with the stand-in sequence (see 5.2).

2. Overview

NanoPoreSeq produces BAM files that imitate adaptive-sampling Nanopore sequencing (NPHD panel, hg38 genome) of a glial tumour. Each BAM contains the known alterations of a tumour profile (variants, copy number, methylation); a ground-truth file describes them. You can thus check that a pipeline such as Rapid-CNS2_nf finds exactly what was simulated.

Contents

Folder	Used to	Scripts
`simulation/`	simulate, check, classify, simulate cohorts	`simulate_glioma_hg38.py, verify_bam.py, build_cohort.py, bedmethyl_from_bam.py, methylation_classification_local.R, 01_export_probe_targets.R, 02_export_cohort_probe_targets.R, run_full_mnp_classification.sh, simnanocns_gui_en.py`
`rapidcns2/`	run Rapid-CNS2_nf; prepare real data	`rapidcns2_gui_en.py, align_fastq_rapidcns2_en.sh`
`usb/`	build a self-contained USB stick	`make_usb_stick_en.sh, prepare_usb_en.sh`
`docs/`	documentation	this guide, review reports

> **Note:** The scripts of one folder call each other: keep them together. The full tree of the project and of the folders produced is described in 8.5.

Typical workflow

1. Simulate one or several BAMs (section 6.1 or 6.4).
2. Check each BAM with verify_bam.py (6.2).
3. Optional: obtain the expected methylation class (6.3).
4. Analyse the BAMs with Rapid-CNS2_nf and compare the report with the ground truth (6.6).

3. Quick start

This test of a few minutes checks that everything works, without an hg38 reference: it simulates a glioblastoma on chromosome 10 only with the stand-in sequence, then checks it.

1. Install the Python dependencies and fetch the Rapid-CNS2_nf data.

```
python3 -m pip install pysam numpy
git clone https://github.com/areebapatel/Rapid-CNS2_nf
export DATA=$PWD/Rapid-CNS2_nf/data
```

2. Simulate (about one minute).

```
cd NanoPoreSeq/simulation
python3 simulate_glioma_hg38.py --profile gbm --standin --data-dir $DATA \
    --out test_gbm --chroms chr10
```

3. Check.

```
python3 verify_bam.py --bam test_gbm/SIM_GBM_HG38.bam --data-dir $DATA --standin
```

4. Read the end of the output: MGMT methylation should be close to 0.88, MGMT coverage above 10× and the mismatch rate close to 2%.

> **Note:** A BAM simulated with --standin is only for testing the chain. For Rapid-CNS2, simulate with the real reference (--ref).

4. Installation

4.1 Software

Component	Needed for	Installation
Python 3, pysam, numpy	everything in Python	`python3 -m pip install pysam numpy`
Streamlit ≥ 1.37, pandas	simulation interface	`python3 -m pip install "streamlit>=1.37" pandas`
tkinter	Rapid-CNS2 interface	`sudo apt install python3-tk`
R and packages	classification, profile export	see below
Nextflow, Java ≥ 17, Docker / Apptainer	Rapid-CNS2_nf	Rapid-CNS2_nf documentation
dorado, samtools	real data (FASTQ)	Oxford Nanopore site; package manager
rsync, curl	USB stick	`sudo apt install rsync curl`

> **Note:** Nextflow 26.04 and later parses files with a strict syntax that Rapid-CNS2_nf 3.0 does not follow yet (error "Unexpected input: 'java'" at start-up). The Rapid-CNS2 interface and the USB stick set NXF_SYNTAX_PARSER=v1 (legacy parser) automatically; on the command line, run export NXF_SYNTAX_PARSER=v1 before nextflow run.

R packages: optparse, ranger, matrixStats, data.table, glmnet and GenomicRanges. On Ubuntu, the system packages are enough (method used for the tests); elsewhere, use CRAN and Bioconductor.

```
sudo apt install r-base-core r-cran-optparse r-cran-ranger r-cran-matrixstats \
    r-cran-data.table r-cran-glmnet r-bioc-genomicranges
# or
Rscript -e
'install.packages(c("optparse","ranger","matrixStats","data.table","glmnet","BiocManager"))'
Rscript -e 'BiocManager::install("GenomicRanges")'
```

4.2 Data and reference

1. Fetch Rapid-CNS2_nf: its data/ folder provides the NPHD panel, the MGMT region, the genes, refGene and the Capper et al. training data.

   ```
   git clone https://github.com/areebapatel/Rapid-CNS2_nf
   export DATA=$PWD/Rapid-CNS2_nf/data
   ```

2. Point to an hg38 FASTA. The contigs must be named chr1… (UCSC) or NC_000001.11… (NCBI RefSeq). The .fai index is created automatically if missing and if the folder is writable.

   ```
   export REF=/path/to/hg38.fa
   ```

4.3 Check

Each command-line script prints its options and their default values with --help. The quick start (section 3) validates the Python installation; section 6.3 validates the R installation.

```
python3 simulate_glioma_hg38.py --help
Rscript methylation_classification_local.R --help
```

5. Key concepts

5.1 The three tumour profiles

Alteration	gbm	astro	oligo
Diagnosis	IDH-wildtype glioblastoma	IDH-mutant astrocytoma	IDH-mutant, 1p/19q-codeleted oligodendroglioma
Chromosomes 7 / 10	gain / loss	normal	normal
1p / 19q	—	—	single-copy loss
EGFR	amplification	—	—
CDKN2A/B	homozygous deletion	grade 4 option (--cdkn2a-del)	—
IDH1 R132H	no	yes	yes
TERT C228T	yes (VAF 0.80)	no	yes
TP53 R175H	—	yes	no
1 bp deletion	—	ATRX	CIC
MGMT promoter	methylated	methylated	methylated
Expected methylation class	GBM_RTK_II	A_IDH (A_IDH_HG in grade 4)	O_IDH

5.2 Real sequence or stand-in sequence

With --ref, the reads are drawn from the real hg38 FASTA: the BAM is usable by Rapid-CNS2. With --standin, they are drawn from a synthetic sequence at the same coordinates: it is quicker to set up and enough to test the scripts, but such a BAM does not match the real genome and must not be given to Rapid-CNS2.

5.3 Purity and allele frequency

The tumour purity (--purity, 0.80 by default) is the share of tumour cells. A heterozygous variant then appears in a fraction purity/2 of the reads (0.40 by default); TERT C228T of the gbm profile is fixed at 0.80. The measured frequency varies from one sample to the next depending on the depth.

5.4 Ground truth

Each simulation writes truth_events.tsv next to the BAM: the exact list of simulated events (section 8.1). It is the reference for checking the BAM and for judging the Rapid-CNS2 report.

5.5 Real methylation profiles (MNP probes)

With no other option, the methylation of the BAM is a generic background that does not allow a realistic classification. With --mnp-targets, the simulator reproduces, at the positions of the HM450 probes used by the classifier, the binarised profile of a real sample of the target class (Capper et al. data). The profile files are produced by 01_export_probe_targets.R (one representative per class) or 02_export_cohort_probe_targets.R (several distinct samples, for cohorts).

5.6 Reproducibility

The same seed (--seed) with the same options and the same version of the scripts gives the same BAM. In a cohort, individual i gets the seed --seed-base + i.

6. How to

6.1 Simulate one sample

1. Choose the profile, the purity and a sample name.
2. Run the simulation on the whole genome (chr1–22 and chrX).

```
python3 simulate_glioma_hg38.py --profile astro --ref $REF --data-dir $DATA \
    --out sim_astro --sample-id ASTRO01 --purity 0.7
```

3. The script first checks the hg38 base of each hotspot, prints the number of reads per chromosome, then "Done". The BAM and its index are in the --out folder, with truth_events.tsv.

Useful variants:

- Grade 4 astrocytoma: add --cdkn2a-del.
- Realistic classification: add --mnp-targets probe_targets/probe_targets_astro.tsv (see 6.3).
- CNV variability: --cnv-jitter-sd 0.15 (about ±15%).
- Quick test: --chroms chr2,chr19 limits the simulation to a few chromosomes.
- Coverage: --ontarget-depth (panel), --offtarget-depth (off-panel, used for CNVs), --mgmt-depth-factor (MGMT promoter).

6.2 Check a sample

```
python3 verify_bam.py --bam sim_astro/ASTRO01.bam --data-dir $DATA --ref $REF
```

With a BAM simulated with --standin, replace --ref $REF with --standin and give the same --seed as the simulation. The output is split into sections:

Section	What is measured	Expected result
`Header / format`	basecalling model, contigs, reads with MM tag	25 contigs; almost all reads have an MM tag
`Mean coverage in the NPHD panel`	panel coverage per chromosome	indicative only
`Copy number`	copy ratio of large events	close to the "expected" value (0.50 for a loss, 1.5 for a gain)
`Focal events`	EGFR, CDKN2A/B	EGFR clearly above the panel, CDKN2A/B almost zero
`Hotspots`	base counts and VAF	VAF close to purity/2 for the variants; a few % at wildtype sites (errors)
`MGMT`	promoter depth and methylation; control	≥ 10×; ~0.88 on both strands; control close to the expected background
`Alignment / sequence`	mismatch rate	~2%

> **Note:** With --chroms, events on chromosomes that were not simulated are reported as "not simulated": this is normal.

6.3 Obtain the methylation class

The simplest method runs the four steps (profile export, simulation with MNP probes, bedMethyl, classification) for one profile:

```
./run_full_mnp_classification.sh --profile astro --data-dir $DATA --ref $REF \
    --num-trees 20000 --threads 16
```

The result is printed at the end and stored in classif_astro/. To classify a BAM already simulated with --mnp-targets:

```
python3 bedmethyl_from_bam.py --bam sim/X.bam --out sim/X.bedmethyl.bed
Rscript methylation_classification_local.R -s X -o classif_X -i sim/X.bedmethyl.bed \
    -p $DATA/top_probes_hm450.Rdata -a $DATA/HM450.hg38.manifest.gencode.v22.Rdata \
    -d $DATA/capper_top_100k_betas_binarised.Rdata -t 16 -n 20000
```

- Read classif_X/X_calibrated_classification.tsv: "Predicted Class" must be the expected class of the profile (section 5.1).
- Number of trees: 20,000 as in production; 2,000 for screening; fewer for a mere test. Memory grows with the number of trees (section 9).
- "glm.fit: algorithm did not converge" warnings are normal (section 10).

6.4 Simulate a cohort

1. Export the real profiles: n_per_class = size of the largest group (33 for 99 individuals in equal shares).

   ```
   Rscript 02_export_cohort_probe_targets.R $DATA cohort_probes 33
   ```

2. Do a reduced trial (a few individuals, one chromosome).

   ```
   python3 build_cohort.py --n 3 --standin --data-dir $DATA --out-dir cohort_test \
       --chroms chr17 --jobs 3 --mnp-targets-dir cohort_probes
   ```

3. Run the full cohort, with or without classification.

```
python3 build_cohort.py --n 99 --ref $REF --data-dir $DATA --out-dir cohort_bams \
    --mnp-targets-dir cohort_probes --jobs 8 --cnv-jitter-sd 0.15 \
    --astro-grade4-fraction 0.3 --classify --classify-jobs 2 --num-trees 2000
```

4. Check cohort_bams/cohort_manifest.tsv (all rows with status OK) and, with --classify, the correct classification rate printed at the end and the summary cohort_classification/cohort_classification_summary.tsv.
   - Split: equal shares by default; --profile-counts gbm=50,astro=30,oligo=19 imposes group sizes (sum = --n).
   - Purity drawn at random within --purity-range (0.60 to 0.95 by default).
   - --jobs sets the number of simultaneous simulations and --classify-jobs that of the classifications: adapt them to the memory (section 9).
   - The A_IDH_HG class has only 46 real samples: beyond that, profiles are reused (a warning says so).

6.5 Simulation interface

1. Start the interface on the computing machine. From another computer, open an SSH tunnel then http://localhost:8501.

```
streamlit run simnanocns_gui_en.py
ssh -L 8501:localhost:8501 <machine>
```

2. Sidebar: working folder, scripts folder (the simulation/ folder), Python interpreter, data/ folder, reference (real hg38 or stand-in sequence). "Save as defaults" stores these settings; "Export / Import (JSON)" lets you share them.
3. Single sample or Cohort tab: set the parameters. The right column shows the exact command and the checks; the run button stays greyed out while an error remains.
4. Run tab: duration, number of BAMs produced, live log, then results (ground truth, manifest, classification rate). The logs are kept in <working folder>/gui_logs/.

> **Note:** The computation continues if the browser tab is closed, but stops if the Streamlit server is stopped. The SimNanoCNS logo appears at the top of the sidebar and as the tab icon (file simnanocns_logo.png next to the script).

6.6 Test Rapid-CNS2 with the simulated BAMs

1. Simulate with --ref (never --standin) and, for an interpretable classification, with --mnp-targets.
2. Simulated data: the interface ticks the "Simulated BAM" box itself when the BAM header carries the "SIMULATED DATA" comment written by the simulator. It then writes .rapidcns2_simulated.config in the launch folder and adds it to the command: failures of AnnotSV (no structural variant, hence no table) and SAVANA CNA are ignored, with no effect on the report. Untick the box for real data. On the command line, the same effect is obtained with this file passed with -c:

```
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
```

3. Open the Rapid-CNS2 interface, choose the cohort folder and the sample, check the paths and options, then "Run". "Copy command" gives the equivalent command for a terminal. The interface automatically mounts the AnnotSV and ANNOVAR folders in the containers (--containerBindPaths); RCNS_BINDPATHS adds other ones, comma-separated. The window shows the SimNanoCNS logo at the top (file simnanocns_logo.png next to the script; on the USB stick, in gui/).

```
cd NanoPoreSeq/rapidcns2
python3 rapidcns2_gui_en.py
```

4. At the end, "Open report" opens the HTML report. Compare it with truth_events.tsv using the table below.

Report item	gbm	astro	oligo
Methylation class	GBM_RTK_II	A_IDH (A_IDH_HG if grade 4)	O_IDH
MGMT status	methylated	methylated	methylated
Copy-number profile	+7, −10, EGFR amplification, CDKN2A/B deletion	flat (CDKN2A/B deletion if grade 4)	1p and 19q loss
Variants	TERT C228T	IDH1 R132H, TP53 R175H	IDH1 R132H, TERT C228T

> **Note:** Also check mgmt/mgmt_avg_cov.txt: if the MGMT coverage is below --minimumMgmtCov (10 in the interface), the MGMT module is skipped.

6.7 Prepare real data

For FASTQ files from real sequencing, align_fastq_rapidcns2_en.sh merges the files, aligns to hg38, sorts, indexes and checks the result.

```
./align_fastq_rapidcns2_en.sh -i fastq_pass/ -r $REF -s GBM01 -o results -t 24
```

> **Note:** Methylation is only available if the FASTQ files carry the MM/ML tags in their headers (for instance produced by samtools fastq -T '*'). A standard MinKNOW FASTQ has none: then start again from the POD5 files or the unaligned BAM.

6.8 Self-contained USB stick

The recommended method is make_usb_stick_en.sh: it checks that the device really is a removable stick (never a system disk), erases it and formats it as ext4 after confirmation, mounts it, then runs prepare_usb_en.sh, which copies the pipeline, Nextflow, Java, the containers, the reference, ANNOVAR, AnnotSV, the interface and the requested BAMs, and ends with run.sh --check. Run it with your usual account: it calls sudo itself for the privileged steps.

1. Plug in the stick and find its name (columns TRAN = usb or RM = 1).

```
cd NanoPoreSeq/usb
./make_usb_stick_en.sh --list
```

2. Simulate the operation without changing anything, then check the space needed (about 100 GB or more depending on the containers and ANNOVAR).

```
./make_usb_stick_en.sh --dry-run /dev/sdX
```

3. Create the stick. The script asks you to type the device name to confirm the erasure; the options placed after "--" are passed to prepare_usb_en.sh. The source paths are set with the PIPELINE_DIR, REF_FA, ANNOVAR_DIR and ANNOTSV_DIR variables.

```
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer \
    --with-samples /path/to/cohort_bams
```

4. On the host computer: check, then start the interface or the command line. The Nextflow work folder goes on the host's disk (allow ~150 GB per sample; changeable with RCNS_WORKDIR).

```
cd /media/$USER/RAPIDCNS2/rapidcns2
./run.sh --check
./run_gui.sh
./run.sh samples/IND001_GBM/IND001_GBM.bam IND001_GBM
```

make_usb_stick_en.sh option	Effect
/dev/sdX	whole disk: new GPT partition table, one ext4 partition
/dev/sdX1	partition: only this partition is reformatted
/media/$USER/RAPIDCNS2	mount folder of an already formatted stick: nothing is erased (as with --keep-format)
--keep-format	keeps the existing ext4 file system and completes the stick (update)
--label NAME	file system label (RAPIDCNS2 by default)
--eject	unmounts and powers off the stick at the end
--yes	no interactive confirmation (the safety checks stay active)
--allow-non-removable	accepts a disk not reported as removable/USB (some SSD enclosures)
--mountpoint DIR	mount point if udisksctl is absent
--dry-run, --list	trial run without changes; list of the disks

Note: Safety: the script refuses any disk with a partition mounted outside /media, /run/media, /mnt (or the --mountpoint folder) or used as swap, which excludes the system disk, even with --allow-non-removable. All content of the stick is erased, except with --keep-format.

Equivalent manual method: format the stick yourself (lsblk, umount, mkfs.ext4 -m 0 -L RAPIDCNS2, replug, chown), then run prepare_usb_en.sh on its mount point (section 12.3).

- --no-annovar: ANNOVAR is not copied (it may not be redistributed); on the host, point RCNS_ANNOVARPATH and RCNS_ANNOVARDB to a local installation.
- Offline, wf-human-variation is disabled (RCNS_HUMANVAR=true to enable it with network access) and automatic upload to MNP-Flex is impossible.
- conf/usb.config ignores AnnotSV and SAVANA failures: comment out these lines for real data.
- The complete code of the two stick scripts is in the appendix, section 12.

7. Command reference

7.1 simulate_glioma_hg38.py

Option	Default	Description
--profile	required	gbm, astro or oligo
--ref FASTA	—	real hg38 FASTA (one of --ref or --standin)
--standin	—	stand-in sequence (tests)
--data-dir	required	data/ folder of Rapid-CNS2_nf
--out	sim_glioma	output folder
--sample-id	SIM_<PROFILE>_HG38	sample and BAM name
--purity	0.80	tumour purity, between 0 and 1

Option	Default	Description
`--ontarget-depth`	8	depth per panel region (effective coverage ~2.5 times higher)
`--offtarget-depth`	2	off-panel depth
`--mgmt-depth-factor`	4	MGMT depth = factor × --ontarget-depth × local copy ratio
`--cdkn2a-del`	—	astro: CDKN2A/B homozygous deletion (grade 4)
`--cnv-jitter-sd`	0	standard deviation of the noise on copy ratios
`--chroms`	chr1–22, chrX	comma-separated list
`--seed`	2026	random seed
`--mnp-targets`	—	real profile file at the HM450 probes
`--mnp-depth`	6	dedicated depth per probe
`--mnp-window`	150	half-window around each probe (bp)
`--skip-ref-check`	—	skips the reference base check

7.2 verify_bam.py

Option	Default	Description
`--bam`	required	simulated BAM
`--data-dir`	required	data/ folder of Rapid-CNS2_nf
`--ref / --standin`	—	as for the simulation
`--truth`	next to the BAM	truth_events.tsv file
`--seed`	2026	seed of the simulation (with --standin)

7.3 build_cohort.py

Option	Default	Description
`--n`	99	number of individuals
`--profile-counts`	equal shares	group sizes, e.g. gbm=50,astro=30,oligo=19
`--ref / --standin, --data-dir`	—	as for the simulation
`--out-dir`	cohort_bams	output folder
`--mnp-targets-dir`	—	real profiles (output of 02_export…); required with --classify and --astro-grade4-fraction
`--mnp-depth`	6	depth per probe
`--chroms`	chr1–22, chrX	restricts the chromosomes
`--purity-range MIN MAX`	0.60 0.95	purity range
`--cnv-jitter-sd`	0	copy-ratio variability
`--astro-grade4-fraction`	0	share of grade 4 astrocytomas
`--seed-base`	2026	seed of individual i = seed-base + i
`--jobs`	4	simulations in parallel
`--classify`	—	classifies each BAM after simulation

Option	Default	Description
`--classify-out-dir`	cohort_classification	classification folder
`--classify-jobs`	2	classifications in parallel
`--classify-threads`	2	R threads per classification
`--num-trees`	2000	random forest trees

7.4 Classification and profile export

Option	Default	Description
`bedmethyl_from_bam.py --bam --out`	required	input BAM and output bedMethyl
`bedmethyl_from_bam.py --min-cov`	1	minimum coverage of a site
`methylation_classification_local.R -s -o -i`	required	name, output folder, bedMethyl
`… -p -a -d`	data/ files	probes, HM450 manifest, training data
`… -t / -n`	16 / 20000	threads / number of trees
`01_export_probe_targets.R <data> <sortie>`	required	one representative profile per class
`02_export_cohort_probe_targets.R <data> <sortie> <n> [graine]`	seed 2026	n distinct profiles per class
`run_full_mnp_classification.sh`	--num-trees 20000, --threads 16	--profile, --data-dir, --ref or --standin

7.5 Rapid-CNS2, real data and USB stick

Option	Default	Description
`align_fastq_rapidcns2_en.sh -i -r -s`	required	FASTQ (file or folder), reference, sample
`… -o / -t / -m`	rapidcns2_input / 16 / —	output folder, threads, minimap2 options
`make_usb_stick_en.sh DEVICE [-- …]`	required	formats, mounts and fills the stick (see 6.8)
`prepare_usb_en.sh <point de montage>`	required	stick formatted as ext4
`… --engine`	apptainer if present, else docker	apptainer, docker or both
`… --no-annovar / --no-java`	—	does not bundle ANNOVAR / Java
`… --with-samples DIR / --dry-run`	—	copies BAMs / trial run without writing
`rapidcns2_gui_en.py`	—	defaults at the top of the script or RCNS_<FIELD> variables

8. File reference

8.1 truth_events.tsv

Columns: event, type, chrom, start, end, value (0-based coordinates, end excluded). Types:

type	Meaning	value
`copy_ratio`	copy ratio of a chromosome or region	e.g. 1.5, 0.5, 9.0, 0.02
`methylation`	methylated MGMT promoter	0.88
`background`	methylated share of the background	0.470 (gbm), 0.642 (astro, oligo)
`snv`	point variant	ref>alt;vaf=…
`wildtype`	hotspot left wildtype	reference base
`del`	1 bp deletion (ATRX or CIC)	vaf=…

8.2 Cohort files

File	Content
`cohort_manifest.tsv`	individual, profile, profile_index, seed, purity, grade4, expected_dx, mnp_file, bam, status, elapsed_s
`INDnnn_<PROFILE>.log`	simulation log of the individual
`cohort_classification_summary.tsv`	individual, profile, expected_dx, predicted_dx, correct, calibrated_score, n_features, status
`cohort_probe_manifest.tsv`	output of 02_export: profile, rank, class, real sample identifier, file, number of probes, % methylated

8.3 Classification outputs

File	Content
`<id>_calibrated_classification.tsv`	Number of features, Predicted Class, Initial Score, Calibrated Score
`<id>_votes.tsv`	frequency (and recalibrated frequency) of each class
`<id>_rf_details.tsv`	covered probes, Brier score, training probes
`<id>_methylation_hg38_HM450.RDS`	sample methylation per HM450 probe
`<id>_probes_for_training.csv`	probes shared by the sample and the reference set

8.4 bedMethyl

18 columns in modkit format; one row per CpG (position of the C on the + strand, strand "."). Only columns 1–3 (position), 6 (strand), 10 (coverage) and 11 (% methylated) are read by the classification.

8.5 Folder trees

Layout of the project and of the folders created by the scripts, with the default names.

Project

Content of NanoPoreSeq.zip. The scripts of simulation/ call each other: they must stay in the same folder.

```
NanoPoreSeq/
├── README.md                                overview of the folders
├── simulation/                              simulate, check, classify
│   ├── simulate_glioma_hg38.py              simulates a BAM
│   ├── verify_bam.py                        checks a BAM
│   ├── build_cohort.py                      simulates a cohort
│   ├── bedmethyl_from_bam.py                BAM → bedMethyl
│   ├── methylation_classification_local.R   classifies the methylation
│   ├── 01_export_probe_targets.R            1 real profile per class
│   ├── 02_export_cohort_probe_targets.R     N real profiles per class
│   ├── run_full_mnp_classification.sh       complete chain
│   ├── simnanocns_gui_en.py                 Streamlit interface
│   └── simnanocns_logo.png                  logo shown by the interface
├── rapidcns2/                               run Rapid-CNS2_nf
│   ├── rapidcns2_gui_en.py                  Tkinter interface
│   ├── align_fastq_rapidcns2_en.sh          FASTQ → BAM (real data)
│   └── simnanocns_logo.png                  logo shown by the interface
├── usb/
│   ├── make_usb_stick_en.sh                 formats and creates the USB stick (all-in-one)
│   └── prepare_usb_en.sh                    copies the content onto the stick
└── docs/                                    guides and reports (FR, EN; .docx and .pdf)
```

In the GitHub repository (badozor/SimNanoCNS), these folders are under trunk/. The logo/ folder, at the root next to trunk/, holds the SimNanoCNS logo (simnanocns_hex.svg, simnanocns_hex.png), its generator make_logo.py and the brain outline brain_contour.npy; python3 logo/make_logo.py regenerates the SVG. Without simnanocns_logo.png next to them, the interfaces look for the logo in logo/ at the repository root, then start without it.

Simulated sample

Output of simulate_glioma_hg38.py (--out sim_gbm, gbm profile).

```
sim_gbm/                                 --out folder
├── SIM_GBM_HG38.bam                     simulated BAM (name = --sample-id)
├── SIM_GBM_HG38.bam.bai                 index
└── truth_events.tsv                     ground truth
```

Classification of one sample

Outputs of 01_export_probe_targets.R and of the classification (procedure of section 6.3).

```
probe_targets/                           01_export_probe_targets.R
└── probe_targets_gbm.tsv, …_astro.tsv, …_oligo.tsv
classif_gbm/                             run_full_mnp_classification.sh (-o)
├── <id>_calibrated_classification.tsv   predicted class, scores
├── <id>_votes.tsv, <id>_rf_details.tsv
├── <id>_methylation_hg38_HM450.RDS
└── <id>_probes_for_training.csv
```

Cohort

Outputs of build_cohort.py and 02_export_cohort_probe_targets.R (default names).

```
cohort_bams/                                  --out-dir folder
├── cohort_manifest.tsv                       one individual per row, status
├── IND001_OLIGO.log                          simulation log
└── IND001_OLIGO/                             one folder per individual
    ├── IND001_OLIGO.bam (+ .bai)
    ├── IND001_OLIGO.bedmethyl.bed            with --classify
    └── truth_events.tsv
cohort_classification/                        with --classify (--classify-out-dir)
├── cohort_classification_summary.tsv         expected / predicted class
└── IND001_OLIGO/
    ├── …_calibrated_classification.tsv       predicted class
    ├── …_votes.tsv, …_rf_details.tsv
    ├── …_methylation_hg38_HM450.RDS
    ├── …_probes_for_training.csv
    └── …_classify.log                        prefix … = IND001_OLIGO
cohort_probes/                                02_export_cohort_probe_targets.R
├── probe_targets_gbm_01.tsv …                one file per individual and class
└── cohort_probe_manifest.tsv                 real sample used for each file
```

Rapid-CNS2 results

With the rapidcns2_gui_en.py interface, whose default output is results_<id> in the launch folder.

```
<launch folder>/results_<id>/                 Rapid-CNS2_nf output
├── report/<id>_Rapid-CNS2_report_full.html   report (and _lite.html)
├── mods/<id>.5mC.bedmethyl                   methylation per CpG
├── mgmt/mgmt_avg_cov.txt                     MGMT coverage
└── pipeline_info/*_trace.txt                 step tracking
<launch folder>/work/                         Nextflow work (~150 GB/sample)
```

USB stick

Created by make_usb_stick_en.sh (or prepare_usb_en.sh on an already formatted stick). The Nextflow work folder is not on the stick: it is created on the host's disk (~/rapidcns2_work by default, changeable with RCNS_WORKDIR).

```
/media/$USER/RAPIDCNS2/rapidcns2/
├── run.sh, run_gui.sh                        launchers
├── env.sh                                    shared environment
├── README.txt
├── bin/nextflow                              self-contained Nextflow
├── jre/                                      Java (unless --no-java)
├── pipeline/                                 Rapid-CNS2_nf
├── containers/sif/ or docker/                tool images
├── refs/                                     hg38.fa + .fai
├── annovar/                                  unless --no-annovar
├── annotsv/
├── conf/usb.config                           stick-specific settings
├── gui/                                      Rapid-CNS2 interface and its logo
├── samples/                                  copied BAMs (--with-samples)
├── results/<id>/                             analysis results
└── nxf_home/                                 Nextflow cache, plugins
```

9. Resources and run times

Values measured on the test machine (7 GB of memory, stand-in sequence). The whole genome was not simulated: do a first run with few jobs before a large cohort.

Operation	Time	Memory
Simulation chr10, without MNP probes	46 s	—
Simulation chr17, without MNP probes	~1 min	0.77 GB
Simulation of 6 chromosomes (~¼ of the genome) with MNP probes	15.5 min	—
bedMethyl of this BAM (951,528 reads)	5.6 min	—
Export 01 of the representative profiles	23 s	—
Classification, 300 trees, 2 threads	1.6 min	4.4 GB
Classification, 2,000 trees	—	more than 7 GB
Cohort of 3 individuals, chr17, with probes and classification (300 trees)	8.3 min	—

- The memory of a simulation grows with the length of the largest simulated chromosome (chr1 is three times longer than chr17).
- For --classify-jobs, allow more than 7 GB per classification with 2,000 trees, more with 20,000.
- Rapid-CNS2: allow ~150 GB of disk per sample for the Nextflow work folder.

10. Troubleshooting

Message or symptom	Cause	Solution
`Specify either --ref hg38.fa or --standin`	no reference, or both	give exactly one of them
`ERROR: hg38 contigs not found in …`	contig names neither chr* nor RefSeq	use a UCSC or NCBI hg38 FASTA
`ERROR: IDH1_R132H … ref base=…, expected C`	reference that is not hg38 (hg19 for instance)	use hg38; --skip-ref-check only if sure of the version
`--chroms: unknown chromosome(s)`	invalid chromosome name	write chr1 … chr22, chrX
verify_bam.py: mismatch rate ~74%	--standin BAM checked with another seed	pass --seed <simulation seed>
FAILED(rc=-9) status in the manifest	process killed, usually for lack of memory	reduce --jobs; see the individual's .log
`WARNING: no probe file found for …`	real profiles not exported or too few	run 02_export… with n = size of the largest group
`--classify and --astro-grade4-fraction require --mnp-targets-dir`	no real profiles	add --mnp-targets-dir
R classification "Killed" (code 137)	not enough memory	reduce --classify-jobs or --num-trees, or add memory
`glm.fit: algorithm did not converge`	perfectly separated scores during recalibration	normal: the predicted class is not affected

Message or symptom	Cause	Solution
Interface: "… does not know: --option"	scripts folder holding an older version	point to NanoPoreSeq/simulation
Rapid-CNS2: MGMT module skipped	MGMT coverage below --minimumMgmtCov	resimulate with a higher --mgmt-depth-factor, or lower the threshold
Rapid-CNS2: annotSv* or savanaCna failure	expected on a simulated BAM	custom.config (section 6.6)
Rapid-CNS2: inconsistent results	BAM simulated with --standin	resimulate with --ref
`Missing output file(s) `…_annotsv.tsv` expected by process `annotSv…` `	simulated BAM: no structural variant, AnnotSV writes no table	tick "Simulated BAM" in the interface (automatic for our BAMs), or -c with the file of section 6.6
`AnnotSV needs … an existing path of the annotations directory`	annotations folder not mounted in the Docker container	updated interface (automatic mount); on the command line: --containerBindPaths AnnotSV_folder,ANNOVAR_folder
`Config parsing failed / Unexpected input: 'java'`	Nextflow ≥ 26.04: strict syntax, not yet followed by Rapid-CNS2_nf 3.0	export NXF_SYNTAX_PARSER=v1 (automatic in the interface and on the stick)
`the stick is formatted as '…'`	stick in FAT32 or exFAT	reformat as ext4 (section 6.8)
`… is mounted on '…': … is used by the system`	system disk or partition mounted outside /media	check the name with --list; unmount the stick if mounted elsewhere
`… is neither USB nor removable`	enclosure not reported as removable	check that it really is the stick, then --allow-non-removable
`… is a folder, not a mount point`	folder that is not the mount point of a stick	give the device (--list) or the folder where the stick is mounted
`no terminal to confirm`	run without a terminal (script, cron)	add --yes
`ANNOVAR database missing or still compressed`	ANNOVAR database absent or still compressed	decompress the databases or use --no-annovar
run.sh --check: ✘ usable container engine	neither Apptainer nor Docker on the host	install one of them; force with RCNS_ENGINE if needed
`no MM:Z: tag in the FASTQ headers`	FASTQ without methylation	start again from the POD5 files or the unaligned BAM
`contig 'chr7' missing`	reference without chr prefix	use an hg38 reference with chr prefix

11. Glossary

Term	Definition
Adaptive sampling	Nanopore sequencing that enriches the regions of a panel in real time (here NPHD).
bedMethyl	methylation table per CpG site (modkit format).
1p/19q codeletion	combined loss of the short arm of chromosome 1 and the long arm of chromosome 19, specific to oligodendroglioma.
Methylation class	tumour category of the Capper et al. classifier (e.g. GBM_RTK_II).
G-CIMP	hypermethylated phenotype of IDH-mutant gliomas.
Hotspot	position where a known variant is simulated or checked (TERT, IDH1, TP53).

Term	Definition
Copy ratio	copy number relative to a diploid genome (1 = normal, 0.5 = single-copy loss).
HM450 probe	CpG site of the Illumina 450K array, used by the classifier.
MM/ML tags	BAM fields carrying the methylation of each CpG of a read.
VAF	variant allele frequency: share of reads carrying the mutant allele.
Ground truth	exact list of simulated events (truth_events.tsv).

12. Appendix: code for generating the USB stick

This appendix gathers all the code needed to produce the stick: the automatic make_usb_stick_en.sh script (12.0), the manual command sequence (12.1), then the prepare_usb_en.sh script (12.2 and 12.3), reproduced verbatim from the usb/ folder.

12.0 Automatic method: usb/make_usb_stick_en.sh

This script performs in one command everything described in section 12.1 (formatting included) and calls prepare_usb_en.sh (section 12.3). It is reproduced here as is; the reference version remains the file.

```
cd NanoPoreSeq/usb
./make_usb_stick_en.sh --list
./make_usb_stick_en.sh --dry-run /dev/sdX
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer --no-annovar
```

```
#!/usr/bin/env bash
# ============================================================================
# make_usb_stick_en.sh: creates a Rapid-CNS2 USB stick from start to finish
#
#   1. checks that the device is a removable USB disk, never a system disk
#   2. asks for confirmation, then ERASES it: one GPT partition formatted as ext4
#      (or only reformats the partition given, or keeps it with --keep-format)
#   3. mounts it and makes the current user its owner
#   4. runs prepare_usb_en.sh (same folder) to copy everything onto it
#   5. runs the stick's run.sh --check, then optionally ejects the stick
#
# Usage:
#   ./make_usb_stick_en.sh --list
#   ./make_usb_stick_en.sh [options] DEVICE [-- prepare_usb_en.sh options]
#
# DEVICE: whole disk (/dev/sdX: new partition table, ONE partition)
#         or partition (/dev/sdX1: only this partition is reformatted)
#         or mount folder of a stick already formatted (/media/$USER/RAPIDCNS2:
#         nothing is erased, as with --keep-format)
#
# Examples:
#   ./make_usb_stick_en.sh --list
#   ./make_usb_stick_en.sh --dry-run /dev/sdb
#   ./make_usb_stick_en.sh /dev/sdb -- --engine apptainer --with-samples ../cohort_bams
#   ./make_usb_stick_en.sh --keep-format --eject /dev/sdb1 -- --no-annovar
#
# Run it as your normal user: sudo is called only for the privileged steps.
# Needs: lsblk, findmnt, wipefs, mkfs.ext4, parted (or sgdisk), sudo;
#        udisksctl is used to mount and eject when available.
#
# Developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.
# ============================================================================
set -euo pipefail

SCRIPT_DIR=$(cd "$(dirname "$(readlink -f "$0")")" && pwd)
PREPARE="$SCRIPT_DIR/prepare_usb_en.sh"
```

```
LABEL="RAPIDCNS2"; KEEP=0; YES=0; ALLOW_NONREM=0; DRY=0; EJECT=0; LIST=0; MP_OPT=""; DEV_ARG=""
PREP_ARGS=()

usage() { sed -n '2,31p' "$0" | sed 's/^# \{0,1\}//'; cat <<EOF
Options:
  --list                  list the disks and their removable/USB status, then exit
  --label NAME            file system label (default: RAPIDCNS2, at most 16 characters)
  --keep-format           keep the existing ext4/xfs/btrfs file system of the partition
  --yes                   no interactive confirmation (the safety checks still apply)
  --allow-non-removable   accept a disk that the system does not report as removable/USB
  --mountpoint DIR        mount point if udisksctl is not available
                          (default: /media/\$USER/LABEL)
  --eject                 unmount and power off the stick at the end
  --dry-run               run the checks and show the actions, without changing anything
  -h, --help              this help
Everything after "--" is passed to prepare_usb_en.sh (e.g. --engine, --no-annovar,
--with-samples DIR, --no-java).
EOF
}

say()  { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
warn() { printf '\033[1;33mWARNING: %s\033[0m\n' "$*" >&2; }
die()  { printf '\033[1;31mERROR: %s\033[0m\n' "$*" >&2; exit 1; }
# runs a command, or only shows it with --dry-run
run()  { if [ "$DRY" = 1 ]; then printf '  [dry-run] %s\n' "$*"; else "$@"; fi; }
# name of the first partition: /dev/sdb -> /dev/sdb1, /dev/nvme0n1 or /dev/loop0 -> ...p1
part1() { case "$1" in *[0-9]) echo "${1}p1" ;; *) echo "${1}1" ;; esac; }

while [ $# -gt 0 ]; do
    case "$1" in
        --list)               LIST=1; shift ;;
        --label)              [ $# -ge 2 ] || die "--label needs a value"; LABEL=$2; shift 2 ;;
        --keep-format)        KEEP=1; shift ;;
        --yes)                YES=1; shift ;;
        --allow-non-removable) ALLOW_NONREM=1; shift ;;
        --mountpoint)         [ $# -ge 2 ] || die "--mountpoint needs a value"; MP_OPT=$2; shift 2 ;;
        --eject)              EJECT=1; shift ;;
        --dry-run)            DRY=1; shift ;;
        -h|--help)            usage; exit 0 ;;
        --)                   shift; PREP_ARGS=("$@"); break ;;
        -*)                   die "unknown option: $1 (see --help)" ;;
        *)                    [ -z "$DEV_ARG" ] || die "only one device can be given"; DEV_ARG=$1; shift ;;
    esac
done

if [ "$LIST" = 1 ]; then
    echo "Disks (TRAN=usb or RM=1: removable; MOUNTPOINT: currently mounted):"
    lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MODEL,TRAN,RM,MOUNTPOINT
    exit 0
fi
[ -n "$DEV_ARG" ] || { usage; exit 1; }
[[ "$LABEL" =~ ^[A-Za-z0-9_-]{1,16}$ ]] || die "--label: 1 to 16 characters among letters, digits, _ and -"
[ -x "$PREPARE" ] || [ -f "$PREPARE" ] || die "prepare_usb_en.sh not found next to this script ($PREPARE)"

# ------------------------------------------------------------------------------ user and sudo
if [ "$(id -u)" = 0 ]; then
    [ -z "${SUDO_USER:-}" ] || die "run this script as your normal user, not with sudo: \
it calls sudo itself, and prepare_usb_en.sh must run as you (Docker/Apptainer, Nextflow)"
    SUDO=(); OWNER=root
else
    command -v sudo >/dev/null || die "sudo is required for formatting and mounting"
    SUDO=(sudo); OWNER=$(id -un)
fi
LOG="make_usb_stick_$(date +%Y%m%d_%H%M%S).log"
[ "$DRY" = 1 ] || exec > >(tee -a "$LOG") 2>&1

# ------------------------------------------------------------------------------ checks
say "Device checks"
for tool in lsblk findmnt wipefs mkfs.ext4; do
    command -v "$tool" >/dev/null || die "$tool not found (package util-linux / e2fsprogs)"
done
DEV=$(readlink -f "$DEV_ARG")
```

```
if [ -d "$DEV" ]; then
    # a folder: accepted only if it is the mount point of a partition, which is then kept
    SRC=$(findmnt -nro SOURCE --mountpoint "$DEV" 2>/dev/null | head -1 || true)
    [ -n "$SRC" ] || die "$DEV_ARG is a folder, not a mount point: give the stick device \
(e.g. /dev/sdb, see --list) or the folder where the stick is mounted"
    echo "$DEV_ARG is the mount point of $SRC: this partition is kept, nothing is erased"
    [ -n "$MP_OPT" ] || MP_OPT=$DEV
    DEV=$(readlink -f "$SRC"); KEEP=1
fi
[ -b "$DEV" ] || die "$DEV_ARG is neither a block device nor a mount point (see --list)"
TYPE=$(lsblk -dno TYPE "$DEV")
case "$TYPE" in
    disk|loop) DISK=$DEV; PART="" ;;
    part)      DISK=/dev/$(lsblk -no PKNAME "$DEV" | head -1); PART=$DEV ;;
    *)         die "$DEV is of type '$TYPE': give a whole disk (/dev/sdX) or a partition (/dev/sdX1)" ;;
esac
[ -b "$DISK" ] || die "cannot find the disk holding $DEV"
if [ "$KEEP" = 1 ] && [ -z "$PART" ]; then
    die "--keep-format needs a partition (e.g. $(part1 "$DISK")), not a whole disk"
fi

# 1) never a disk used by the system: any mount outside /media, /run/media, /mnt and the
#    --mountpoint folder, or any swap, means the disk belongs to the system
MP_ABS=""; [ -z "$MP_OPT" ] || MP_ABS=$(readlink -m "$MP_OPT")
while read -r name mp; do
    [ -n "$mp" ] || continue
    mp=$(printf '%b' "$mp")                    # lsblk -r escapes spaces as \x20
    case "$mp" in
        /media/*|/run/media/*|/mnt|/mnt/*) continue ;;
    esac
    [ -n "$MP_ABS" ] && { [ "$mp" = "$MP_ABS" ] || [ "${mp#"$MP_ABS"/}" != "$mp" ]; } && continue
    die "$name is mounted on '$mp': $DISK is used by the system, it will not be touched"
done < <(lsblk -nrpo NAME,MOUNTPOINT "$DISK")

# 2) a removable / USB disk, unless explicitly overridden
RM=$(lsblk -dno RM "$DISK" | tr -d ' ')
TRAN=$(lsblk -dno TRAN "$DISK" | tr -d ' ')
if [ "$TRAN" != usb ] && [ "$RM" != 1 ]; then
    [ "$ALLOW_NONREM" = 1 ] || die "$DISK is neither USB nor removable (TRAN='${TRAN:-?}', RM=$RM). \
If you are sure, add --allow-non-removable"
    warn "$DISK is neither USB nor removable: accepted (--allow-non-removable)"
fi

SIZE_B=$(lsblk -dbno SIZE "$DISK" | tr -d ' ')
SIZE_H=$(lsblk -dno SIZE "$DISK" | tr -d ' ')
[ "$SIZE_B" -ge $((64 * 1024 ** 3)) ] || warn "$DISK is only $SIZE_H: a complete stick needs ~100 GB or more \
(run prepare_usb_en.sh --dry-run to know the exact size)"
lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MODEL,TRAN,MOUNTPOINT "$DISK"

if [ "$KEEP" = 1 ]; then
    # file system type: lsblk (udev), else findmnt if mounted, else blkid
    FST=$(lsblk -dno FSTYPE "$PART" | tr -d ' ')
    [ -n "$FST" ] || FST=$(findmnt -nro FSTYPE "$PART" 2>/dev/null | head -1 || true)
    [ -n "$FST" ] || FST=$("${SUDO[@]}" blkid -o value -s TYPE "$PART" 2>/dev/null || true)
    case "$FST" in ext4|ext3|xfs|btrfs|f2fs) echo "Existing file system kept: $FST" ;;
        *) die "$PART is formatted as '${FST:-none}': Nextflow needs ext4/xfs/btrfs (drop --keep-format)" ;;
    esac
    ACTION="keep $PART ($FST) and add Rapid-CNS2 to it"
elif [ -n "$PART" ]; then
    ACTION="ERASE the partition $PART and format it as ext4 (label $LABEL)"
else
    ACTION="ERASE the whole disk $DISK ($SIZE_H): one GPT partition, ext4, label $LABEL"
fi

# ------------------------------------------------------------------------- confirmation
say "Planned action: $ACTION"
if [ "$DRY" = 0 ] && [ "$KEEP" = 0 ] && [ "$YES" = 0 ]; then
    [ -t 0 ] || die "no terminal to confirm: add --yes to run without confirmation"
    printf 'ALL DATA on %s will be lost. Type the device name (%s) to confirm: ' "$DISK" "$(basename "$DISK")"
    read -r answer
    [ "$answer" = "$(basename "$DISK")" ] || die "confirmation not given: nothing was changed"
fi
```

```
[ "$DRY" = 1 ] || "${SUDO[@]}" true    # asks for the sudo password once, before the long steps

# ------------------------------------------------------------------- unmount the stick
say "Unmounting"
while read -r name mp; do
    [ -n "$mp" ] || continue
    if command -v udisksctl >/dev/null && [ "$DRY" = 0 ] && udisksctl unmount -b "$name" >/dev/null 2>&1; then
        echo "  $name unmounted"
    else
        run "${SUDO[@]}" umount "$name"
    fi
done < <(lsblk -nrpo NAME,MOUNTPOINT "$DISK" | if [ "$KEEP" = 1 ]; then grep -v "^$PART "; else cat; fi)

# ------------------------------------------------------------------- partition and format
if [ "$KEEP" = 0 ]; then
    if [ -z "$PART" ]; then
        say "New partition table on $DISK"
        while read -r child; do run "${SUDO[@]}" wipefs -aq "$child"; done < <(lsblk -nrpo NAME,TYPE "$DISK" | awk
'$2=="part"{print $1}')
        run "${SUDO[@]}" wipefs -aq "$DISK"
        if command -v parted >/dev/null; then
            run "${SUDO[@]}" parted -s "$DISK" mklabel gpt mkpart "$LABEL" ext4 1MiB 100%
        elif command -v sgdisk >/dev/null; then
            run "${SUDO[@]}" sgdisk -Z -n 1:1MiB:0 -t 1:8300 -c "1:$LABEL" "$DISK"
        else
            die "parted or sgdisk is required to partition the disk (sudo apt install parted)"
        fi
        if [ "$DRY" = 1 ]; then
            PART="<first partition of $DISK>"
        else
            command -v partprobe >/dev/null && "${SUDO[@]}" partprobe "$DISK" 2>/dev/null || true
            command -v udevadm >/dev/null && "${SUDO[@]}" udevadm settle 2>/dev/null || true
            for _ in $(seq 1 20); do
                PART=$(lsblk -nrpo NAME,TYPE "$DISK" | awk '$2=="part"{print $1; exit}')
                [ -n "$PART" ] && [ -b "$PART" ] && break
                sleep 0.5
            done
            [ -n "$PART" ] && [ -b "$PART" ] || die "the new partition of $DISK did not appear: unplug and replug the
stick, then rerun with its partition (e.g. $(part1 "$DISK"))"
        fi
    fi
    say "Formatting $PART as ext4 (label $LABEL)"
    run "${SUDO[@]}" wipefs -aq "$PART"
    run "${SUDO[@]}" mkfs.ext4 -F -q -m 0 -L "$LABEL" "$PART"
fi

# ------------------------------------------------------------------------------- mount
say "Mounting"
MP=""
if [ "$DRY" = 1 ]; then
    MP=${MP_OPT:-/media/$OWNER/$LABEL}
    run mount "$PART" "$MP"
    echo "  (udisksctl, if present, may choose another mount point)"
else
    MP=$(findmnt -nro TARGET "$PART" 2>/dev/null | head -1 || true)
    if [ -z "$MP" ] && [ -z "$MP_OPT" ] && command -v udisksctl >/dev/null; then
        out=$(udisksctl mount -b "$PART" 2>/dev/null || true)       # "Mounted /dev/sdb1 at /media/u/LABEL"
        MP=$(printf '%s' "$out" | sed -n 's/^Mounted .* at \(.*\)$/\1/p' | sed 's/\.$//')
    fi
    if [ -z "$MP" ]; then
        MP=${MP_OPT:-/media/$OWNER/$LABEL}
        "${SUDO[@]}" mkdir -p "$MP"
        "${SUDO[@]}" mount "$PART" "$MP"
    fi
    [ -d "$MP" ] || die "mounting $PART failed"
fi
[ "$DRY" = 1 ] || echo "  $PART mounted on $MP"
run "${SUDO[@]}" chown "$OWNER": "$MP"

# ------------------------------------------------------------------- content of the stick
say "Copying Rapid-CNS2 onto the stick (prepare_usb_en.sh)"
if [ "$DRY" = 1 ]; then
```

```
    run bash "$PREPARE" "${PREP_ARGS[@]}" "$MP"
    echo "  (to estimate the space needed: $PREPARE --dry-run <any ext4 mount point>)"
else
    bash "$PREPARE" "${PREP_ARGS[@]}" "$MP"
fi

say "Check of the stick"
if [ "$DRY" = 1 ]; then
    run "$MP/rapidcns2/run.sh" --check
else
    "$MP/rapidcns2/run.sh" --check || warn "some items are missing on the stick (see above)"
    sync
fi

# ------------------------------------------------------------------------------ end
if [ "$EJECT" = 1 ]; then
    say "Ejecting"
    if command -v udisksctl >/dev/null && [ "$DRY" = 0 ]; then
        udisksctl unmount -b "$PART" && udisksctl power-off -b "$DISK" || warn "eject failed: unmount it manually"
    else
        run "${SUDO[@]}" umount "$MP"
    fi
    echo "The stick can be removed."
elif [ "$DRY" = 0 ]; then
    echo
    echo "Stick ready: $MP/rapidcns2   (unmount before removing it: umount $MP)"
fi
[ "$DRY" = 1 ] && echo "--dry-run: nothing was changed." || echo "Log: $LOG"
```

12.1 Manual method: command sequence

To be run on the machine that holds the pipeline, the reference, ANNOVAR and AnnotSV (P3Ultra). Replace /dev/sdX1 with the stick partition shown by lsblk: formatting erases the stick.

```
# 1. Identify and format the stick as ext4
lsblk
sudo umount /dev/sdX1
sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
# 2. Replug the stick, then take ownership of it
sudo chown $USER: /media/$USER/RAPIDCNS2
# 3. (Optional) set the sources if they differ from the defaults
export PIPELINE_DIR=/export/scratch3/gitProject/Rapid-CNS2_nf
export REF_FA=/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa
export ANNOVAR_DIR=/export/scratch3/data/Annotations/humandb/annovar
export ANNOTSV_DIR=/export/scratch3/gitProject/AnnotSV/share/AnnotSV
# 4. Check the space needed, without writing anything
cd NanoPoreSeq/usb
./prepare_usb_en.sh --dry-run /media/$USER/RAPIDCNS2
# 5. Build the stick (add --no-annovar if it is handed to a third party)
./prepare_usb_en.sh --engine apptainer --with-samples /path/to/cohort_bams \
    /media/$USER/RAPIDCNS2
# 6. Check on the host computer
/media/$USER/RAPIDCNS2/rapidcns2/run.sh --check
```

12.2 What the script does

1. Checks: ext4/xfs/btrfs file system, write permission, presence of the pipeline, the reference and its index, AnnotSV, ANNOVAR, nextflow, rsync and a container engine.
2. List of container images read from the pipeline configuration (fallback list if none is found).
3. Estimate of the space needed and comparison with the free space; stops here with --dry-run.
4. Copy of Nextflow (downloaded self-contained binary, else the local launcher) and Java, then of the pipeline and its Nextflow plugins.
5. Containers: .img images for Apptainer/Singularity and/or Docker archives.

6. Copy of the reference, AnnotSV, ANNOVAR (databases of the default protocol), the interface and the requested BAMs.
7. Writing of the env.sh, run.sh, run_gui.sh launchers, conf/usb.config and README.txt.

12.3 Complete script: usb/prepare_usb_en.sh

The script is reproduced here as is; the reference version remains the usb/prepare_usb_en.sh file. The stick launchers (env.sh, run.sh, run_gui.sh, usb.config) are written by the "cat > … <<EOF" blocks of the "Launchers and configuration" section.

```
#!/usr/bin/env bash
# Developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.
# ============================================================================
# prepare_usb_en.sh: builds a self-contained USB stick for Rapid-CNS2_nf
#
# The stick holds: the pipeline, Nextflow (+ Java), the containers, the hg38
# reference, ANNOVAR (+ the 5 databases of the default protocol), the AnnotSV
# annotations, the graphical interface and launchers (run.sh, run_gui.sh).
# Nextflow's "work" directory stays on the host's disk.
#
# The graphical interface (rapidcns2_gui_en.py) is looked for next to this script,
# then in ../rapidcns2/ (layout of the NanoPoreSeq project); or set GUI_PY.
#
# Usage: ./prepare_usb_en.sh [options] /media/$USER/MY_STICK
# Run with --dry-run first to check the sizes.
# ============================================================================
set -euo pipefail

# ----------------------------------------------------------------------------
# Sources (P3Ultra machine). Edit here or override with environment variables.
# ----------------------------------------------------------------------------
PIPELINE_DIR=${PIPELINE_DIR:-/export/scratch3/gitProject/Rapid-CNS2_nf}
REF_FA=${REF_FA:-/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa}
ANNOVAR_DIR=${ANNOVAR_DIR:-/export/scratch3/data/Annotations/humandb/annovar}
ANNOVAR_DB=${ANNOVAR_DB:-$ANNOVAR_DIR/humandb}
ANNOTSV_DIR=${ANNOTSV_DIR:-/export/scratch3/gitProject/AnnotSV/share/AnnotSV}
SCRIPT_DIR=$(cd "$(dirname "$(readlink -f "$0")")" && pwd)
if [ -z "${GUI_PY:-}" ]; then
    GUI_PY=$SCRIPT_DIR/rapidcns2_gui_en.py
    [ -f "$GUI_PY" ] || GUI_PY=$SCRIPT_DIR/../rapidcns2/rapidcns2_gui_en.py
fi

# ANNOVAR databases of the default protocol (refGene,cytoBand,clinvar_20240917,avsnp151,1000g2015aug_eur)
ANNOVAR_REQUIRED=(hg38_refGene.txt hg38_refGeneMrna.fa hg38_cytoBand.txt
                  hg38_clinvar_20240917.txt hg38_avsnp151.txt hg38_EUR.sites.2015_08.txt)
ANNOVAR_OPTIONAL=(hg38_refGeneVersion.txt hg38_clinvar_20240917.txt.idx
                  hg38_avsnp151.txt.idx hg38_EUR.sites.2015_08.txt.idx)

# Fallback images if they cannot be read from the pipeline (README v3.0.x)
FALLBACK_IMAGES=(areebapatel/rapid_cns:3.0.2
                 quay.io/biocontainers/ont-modkit:0.6.4--h7f49ad2_0
                 hkubal/clair3:v2.0.2
                 quay.io/biocontainers/severus:1.7--pyhdfd78af_0
                 quay.io/biocontainers/savana:1.3.8--pyhdfd78af_0)

# ----------------------------------------------------------------------------
usage() {
    cat <<EOF
Usage: $0 [options] USB_STICK_MOUNT_POINT

Options:
  --engine apptainer|docker|both  container format on the stick
                                  (default: apptainer if installed, otherwise docker)
  --no-annovar                    do not copy ANNOVAR (stick handed to a third party:
                                  ANNOVAR may not be redistributed)
  --with-samples DIR              also copy the BAMs (+ .bai) from DIR into samples/
  --no-java                       do not bundle Java (the host must have Java >= 17)
  --dry-run                       show what would be copied and the sizes, without writing
  -h, --help                      this help
```

```
The stick must be formatted as ext4 (or xfs/btrfs), not exFAT/FAT32.
EOF
}

say()  { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
warn() { printf '\033[1;33mWARNING: %s\033[0m\n' "$*" >&2; }
die()  { printf '\033[1;31mERROR: %s\033[0m\n' "$*" >&2; exit 1; }
human() { numfmt --to=iec --suffix=B "$1" 2>/dev/null || echo "$1 B"; }
sz()    { du -sbL "$@" 2>/dev/null | awk '{s+=$1} END {print s+0}'; }
# File name Nextflow uses in its Singularity/Apptainer cache
sif_name() { local n=${1#docker://}; n=${n//\//-}; n=${n//:/-}; echo "$n.img"; }

ENGINE=""; WITH_ANNOVAR=1; SAMPLES_DIR=""; WITH_JAVA=1; DRY=0; USB=""
while [ $# -gt 0 ]; do
    case "$1" in
        --engine)       [ $# -ge 2 ] || die "--engine needs a value"; ENGINE=$2; shift 2 ;;
        --no-annovar)   WITH_ANNOVAR=0; shift ;;
        --with-samples) [ $# -ge 2 ] || die "--with-samples needs a directory"
                        SAMPLES_DIR=$(readlink -f "$2"); shift 2 ;;
        --no-java)      WITH_JAVA=0; shift ;;
        --dry-run)      DRY=1; shift ;;
        -h|--help)      usage; exit 0 ;;
        -*)             die "unknown option: $1 (see --help)" ;;
        *)              USB=$(readlink -f "$1"); shift ;;
    esac
done
[ -n "$USB" ] || { usage; exit 1; }
R="$USB/rapidcns2"

# ------------------------------------------------------------------------------
say "Checks"
# ------------------------------------------------------------------------------
[ -d "$USB" ] || die "$USB does not exist (stick not mounted?)"
FSTYPE=$(findmnt -no FSTYPE --target "$USB" 2>/dev/null || echo unknown)
case "$FSTYPE" in
    ext4|ext3|xfs|btrfs|f2fs) echo "File system: $FSTYPE ✔" ;;
    *) if [ "${RCNS_SKIP_FS_CHECK:-0}" = 1 ]; then warn "file system $FSTYPE accepted (RCNS_SKIP_FS_CHECK=1)"
       else die "the stick is formatted as '$FSTYPE'. Nextflow needs symbolic links and Unix permissions,
       and FAT32 limits files to 4 GB. Reformat it as ext4 (ERASES ALL ITS CONTENT):
         lsblk                                # identify the stick, e.g. /dev/sdX1
         sudo umount /dev/sdX1
         sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
         # replug the stick, then: sudo chown \$USER: /media/\$USER/RAPIDCNS2"
       fi ;;
esac
[ "$DRY" = 1 ] || [ -w "$USB" ] || die "no write permission on $USB: sudo chown $USER: $USB"
[ -z "$SAMPLES_DIR" ] || [ -d "$SAMPLES_DIR" ] || die "--with-samples: directory not found: $SAMPLES_DIR"

[ -f "$PIPELINE_DIR/main.nf" ] || die "pipeline not found: $PIPELINE_DIR/main.nf"
[ -f "$REF_FA" ]               || die "reference not found: $REF_FA"
[ -f "$REF_FA.fai" ]           || die "missing index: samtools faidx $REF_FA"
[ -d "$ANNOTSV_DIR/Annotations_Human" ] || die "Annotations_Human missing from $ANNOTSV_DIR"
[ -f "$GUI_PY" ]               || warn "graphical interface not found ($GUI_PY): it will not be copied"
command -v nextflow >/dev/null || die "nextflow not in the PATH"
command -v rsync    >/dev/null || die "rsync required (sudo apt install rsync)"

if [ "$WITH_ANNOVAR" = 1 ]; then
    for f in "${ANNOVAR_REQUIRED[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] || die "ANNOVAR database missing or still compressed: $ANNOVAR_DB/$f"
    done
    [ -f "$ANNOVAR_DIR/table_annovar.pl" ] || die "table_annovar.pl not found in $ANNOVAR_DIR"
fi

if [ -z "$ENGINE" ]; then
    if command -v apptainer >/dev/null || command -v singularity >/dev/null; then ENGINE=apptainer
    elif command -v docker >/dev/null; then ENGINE=docker
    else die "neither Apptainer/Singularity nor Docker on this machine"; fi
fi
case "$ENGINE" in apptainer|docker|both) ;; *) die "--engine: apptainer, docker or both" ;; esac
APPT=$(command -v apptainer || command -v singularity || true)
if [ "$ENGINE" = apptainer ] || [ "$ENGINE" = both ]; then
    [ -n "$APPT" ] || die "apptainer/singularity not found to build the .sif images"
```

```
fi
if [ "$ENGINE" = docker ] || [ "$ENGINE" = both ]; then
    command -v docker >/dev/null || die "docker not found"
fi
echo "Container format: $ENGINE"

# ------------------------------------------------------------------------------
say "Container images used by the pipeline"
# ------------------------------------------------------------------------------
mapfile -t IMAGES < <(
    grep -rhoE "container[[:space:]]*=?[[:space:]]*['\"][^'\"]+['\"]" \
        "$PIPELINE_DIR"/nextflow.config "$PIPELINE_DIR"/*.nf \
        "$PIPELINE_DIR"/conf "$PIPELINE_DIR"/nextflow 2>/dev/null \
    | sed -E "s/.*['\"]([^'\"]+)['\"]$/\1/" | sed 's#^docker://##' \
    | grep -v '[$]' | grep -E '^[a-zA-Z0-9]' | sort -u )
if [ "${#IMAGES[@]}" -eq 0 ]; then
    warn "no image read from the pipeline, using the README list"
    IMAGES=("${FALLBACK_IMAGES[@]}")
fi
if grep -rqE "container[[:space:]]*=?[[:space:]]*['\"][^'\"]*[$]" \
        "$PIPELINE_DIR"/nextflow.config "$PIPELINE_DIR"/conf "$PIPELINE_DIR"/nextflow 2>/dev/null; then
    warn "some images are defined through a variable and could not be listed: check with run.sh --check"
fi
printf '  %s\n' "${IMAGES[@]}"

# ------------------------------------------------------------------------------
say "Estimating the required space"
# ------------------------------------------------------------------------------
S_PIPE=$(sz "$PIPELINE_DIR")
S_REF=$(sz "$REF_FA"*)
S_ASV=$(sz "$ANNOTSV_DIR")
S_ANV=0
if [ "$WITH_ANNOVAR" = 1 ]; then
    files=(); for f in "${ANNOVAR_REQUIRED[@]}" "${ANNOVAR_OPTIONAL[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] && files+=("$ANNOVAR_DB/$f"); done
    S_ANV=$(sz "${files[@]}" "$ANNOVAR_DIR"/*.pl)
fi
S_SMP=0; [ -n "$SAMPLES_DIR" ] && S_SMP=$(sz "$SAMPLES_DIR")
S_IMG=0
if command -v docker >/dev/null; then
    for i in "${IMAGES[@]}"; do
        s=$(docker image inspect -f '{{.Size}}' "$i" 2>/dev/null || echo 2000000000)
        S_IMG=$((S_IMG + s))
    done
else
    S_IMG=$(( ${#IMAGES[@]} * 2000000000 ))   # ~2 GB per image, estimate
fi
[ "$ENGINE" = both ] && S_IMG=$((S_IMG * 2))
S_JAVA=$(( WITH_JAVA * 400000000 ))
TOTAL=$((S_PIPE + S_REF + S_ASV + S_ANV + S_SMP + S_IMG + S_JAVA + 500000000))
AVAIL=$(df -B1 --output=avail "$USB" | tail -1 | tr -d ' ')
printf '  %-28s %10s\n' "Pipeline" "$(human "$S_PIPE")" "hg38 reference" "$(human "$S_REF")" \
       "ANNOVAR" "$(human "$S_ANV")" "AnnotSV" "$(human "$S_ASV")" \
       "Containers (approx.)" "$(human "$S_IMG")" "Java + Nextflow" "$(human "$S_JAVA")" \
       "Samples" "$(human "$S_SMP")"
printf '  %-28s %10s\n' "TOTAL (approx.)" "$(human "$TOTAL")" "Free on the stick" "$(human "$AVAIL")"
[ "$TOTAL" -lt "$AVAIL" ] || warn "free space looks insufficient"
if [ "$DRY" = 1 ]; then echo; echo "--dry-run: nothing was written."; exit 0; fi
[ "$TOTAL" -lt "$AVAIL" ] || die "not enough space on the stick"

mkdir -p "$R"/{bin,pipeline,refs,annotsv,containers/sif,containers/docker,nxf_home,conf,gui,samples,results}

# ------------------------------------------------------------------------------
say "Nextflow and Java"
# ------------------------------------------------------------------------------
NF_VER=$(nextflow -version 2>/dev/null | grep -oE 'version [0-9]+\.[0-9]+\.[0-9]+' | awk '{print $2}' | head -1 || true)
echo "Nextflow version on this machine: ${NF_VER:-unknown}"
# The self-contained binary (all dependencies included) is ideal offline. It is called
# "-dist" in recent releases and "-all" in older ones: both are tried.
NF_OK=0
if [ -n "$NF_VER" ]; then
    for suffix in dist all; do
```

```
        if curl -fsSL -o "$R/bin/nextflow.tmp" \
                "https://github.com/nextflow-io/nextflow/releases/download/v${NF_VER}/nextflow-${NF_VER}-${suffix}"; then
            mv "$R/bin/nextflow.tmp" "$R/bin/nextflow"
            echo "nextflow-${NF_VER}-${suffix} downloaded ✔"
            NF_OK=1; break
        fi
    done
fi
if [ "$NF_OK" = 0 ]; then
    rm -f "$R/bin/nextflow.tmp"
    warn "could not download the self-contained binary: copying the local launcher"
    cp "$(readlink -f "$(command -v nextflow)")" "$R/bin/nextflow"
fi
chmod +x "$R/bin/nextflow"

if [ "$WITH_JAVA" = 1 ]; then
    JAVA_BIN=$(readlink -f "$(command -v java)" 2>/dev/null || true)
    if [ -n "$JAVA_BIN" ]; then
        JHOME=$(dirname "$(dirname "$JAVA_BIN")")
        rsync -a --delete "$JHOME/" "$R/jre/"
        echo "Java copied from $JHOME ✔"
    else
        warn "java not found: the host will have to provide Java >= 17"
    fi
fi

# ------------------------------------------------------------------------------
say "Pipeline"
# ------------------------------------------------------------------------------
rsync -a --delete --exclude 'work/' --exclude '.nextflow*' --exclude 'results*/' \
      "$PIPELINE_DIR/" "$R/pipeline/"
PIPE_COMMIT=$(git -C "$PIPELINE_DIR" rev-parse --short HEAD 2>/dev/null || echo "n/a")
echo "Rapid-CNS2_nf copied (commit $PIPE_COMMIT) ✔"

# Nextflow plugins the pipeline may declare, cached on the stick
mapfile -t PLUGINS < <(grep -hoE "id[[:space:]]+['\"]nf-[^'\"]+['\"]" "$R/pipeline/nextflow.config" 2>/dev/null \
                       | sed -E "s/.*['\"](nf-[^'\"]+)['\"]/\1/" | sort -u)
(
    export NXF_HOME="$R/nxf_home"
    [ -x "$R/jre/bin/java" ] && export JAVA_HOME="$R/jre"
    "$R/bin/nextflow" -version >/dev/null 2>&1 || warn "first Nextflow launch failed"
    for p in "${PLUGINS[@]}"; do
        "$R/bin/nextflow" plugin install "$p" && echo "plugin $p ✔" || warn "plugin $p not installed"
    done
)

# ------------------------------------------------------------------------------
say "Containers"
# ------------------------------------------------------------------------------
: > "$R/containers/images.tsv"
for img in "${IMAGES[@]}"; do
    if [ "$ENGINE" = apptainer ] || [ "$ENGINE" = both ]; then
        out="$R/containers/sif/$(sif_name "$img")"
        if [ -s "$out" ]; then echo "already present: $(basename "$out")"
        else
            echo "building $(basename "$out")"
            if command -v docker >/dev/null && docker image inspect "$img" >/dev/null 2>&1; then
                "$APPT" build "$out.tmp" "docker-daemon://$img" || "$APPT" pull "$out.tmp" "docker://$img"
            else
                "$APPT" pull "$out.tmp" "docker://$img"
            fi
            mv "$out.tmp" "$out"
        fi
    fi
    if [ "$ENGINE" = docker ] || [ "$ENGINE" = both ]; then
        tgz="$(sif_name "$img")"; tgz="${tgz%.img}.tar.gz"
        if [ -s "$R/containers/docker/$tgz" ]; then echo "already present: $tgz"
        else
            docker image inspect "$img" >/dev/null 2>&1 || docker pull "$img"
            echo "exporting $img"
            docker save "$img" | gzip -1 > "$R/containers/docker/$tgz.tmp"
            mv "$R/containers/docker/$tgz.tmp" "$R/containers/docker/$tgz"
```

```
        fi
        printf '%s\t%s\n' "$img" "$tgz" >> "$R/containers/images.tsv"
    fi
done

# ------------------------------------------------------------------------------
say "Reference, AnnotSV, ANNOVAR"
# ------------------------------------------------------------------------------
REF_BASE=$(basename "$REF_FA")
rsync -aL --info=progress2 "$REF_FA" "$REF_FA".* "$R/refs/" 2>/dev/null || rsync -aL "$REF_FA"* "$R/refs/"
rsync -a --info=progress2 "$ANNOTSV_DIR/" "$R/annotsv/"
if [ "$WITH_ANNOVAR" = 1 ]; then
    mkdir -p "$R/annovar/humandb"
    rsync -a "$ANNOVAR_DIR"/*.pl "$R/annovar/"
    for f in "${ANNOVAR_REQUIRED[@]}" "${ANNOVAR_OPTIONAL[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] && rsync -aL --info=progress2 "$ANNOVAR_DB/$f" "$R/annovar/humandb/"
    done
else
    echo "ANNOVAR not copied (--no-annovar): users will have to point to their own installation"
fi

# ------------------------------------------------------------------------------
say "Graphical interface and samples"
# ------------------------------------------------------------------------------
[ -f "$GUI_PY" ] && cp "$GUI_PY" "$R/gui/rapidcns2_gui.py"
# logo shown by the interface (optional): looked for next to the interface script
GUI_LOGO=$(dirname "$GUI_PY")/simnanocns_logo.png
[ -f "$GUI_LOGO" ] && cp "$GUI_LOGO" "$R/gui/simnanocns_logo.png"
if [ -n "$SAMPLES_DIR" ]; then
    rsync -a --info=progress2 --include='*/' --include='*.bam' --include='*.bai' --exclude='*' \
          --prune-empty-dirs "$SAMPLES_DIR/" "$R/samples/"
fi

# ------------------------------------------------------------------------------
say "Launchers and configuration"
# ------------------------------------------------------------------------------
cat > "$R/env.sh" <<EOF
# shellcheck shell=bash
# env.sh: shared environment, sourced by run.sh and run_gui.sh
R=\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)
export RCNS_USB="\$R"
export PATH="\$R/bin:\$PATH"
if [ -x "\$R/jre/bin/java" ]; then export JAVA_HOME="\$R/jre"; export PATH="\$JAVA_HOME/bin:\$PATH"; fi
export NXF_HOME="\$R/nxf_home"
export NXF_OFFLINE=true
export NXF_DISABLE_CHECK_LATEST=true
# Nextflow >= 26.04 uses the strict syntax by default, not yet followed by Rapid-CNS2_nf 3.0:
# legacy parser unless already set (NXF_SYNTAX_PARSER=v2 once the pipeline is migrated)
export NXF_SYNTAX_PARSER="\${NXF_SYNTAX_PARSER:-v1}"
export NXF_SINGULARITY_CACHEDIR="\$R/containers/sif"
export NXF_APPTAINER_CACHEDIR="\$R/containers/sif"

# Container engine: can be forced with RCNS_ENGINE=singularity|docker
if [ -z "\${RCNS_ENGINE:-}" ]; then
    if { command -v apptainer || command -v singularity; } >/dev/null 2>&1 \\
       && compgen -G "\$R/containers/sif/*.img" >/dev/null; then RCNS_ENGINE=singularity
    elif command -v docker >/dev/null 2>&1 && compgen -G "\$R/containers/docker/*.tar.gz" >/dev/null; then
        RCNS_ENGINE=docker
    fi
fi
export RCNS_ENGINE="\${RCNS_ENGINE:-none}"

# Values read by the graphical interface (RCNS_ prefix + field name in upper case)
export RCNS_PIPELINE="\$R/pipeline/main.nf"
export RCNS_COHORT="\${RCNS_COHORT:-\$R/samples}"
export RCNS_RUNDIR="\${RCNS_WORKDIR:-\$HOME/rapidcns2_work}"   # work directory: host disk
export RCNS_OUTDIR="\${RCNS_OUTDIR:-\$R/results/{id}}"
export RCNS_CONFIG="\$R/conf/usb.config"
export RCNS_REF="\$R/refs/$REF_BASE"
export RCNS_ANNOVARPATH="\${RCNS_ANNOVARPATH:-\$R/annovar}"
export RCNS_ANNOVARDB="\${RCNS_ANNOVARDB:-\$R/annovar/humandb}"
export RCNS_ANNOTSVANNOT="\$R/annotsv"
```

```
export RCNS_PROFILE="local,\$RCNS_ENGINE"
export RCNS_BINDPATHS="\$R"
# wf-human-variation starts a nested Nextflow run that downloads its workflow and
# containers: impossible offline, so it is disabled by default on the stick.
export RCNS_HUMANVAR="\${RCNS_HUMANVAR:-false}"
export RCNS_MAXCPUS="\${RCNS_MAXCPUS:-\$(nproc)}"
export RCNS_MAXMEMORY="\${RCNS_MAXMEMORY:-\$(awk '/MemTotal/ {printf "%d", \$2/1048576*0.8}' /proc/meminfo)}"

rcns_check_engine() {
    if [ "\$RCNS_ENGINE" = none ]; then
        echo "ERROR: no usable container engine (Apptainer/Singularity or Docker)" >&2
        echo "       for the containers stored on the stick." >&2
        return 1
    fi
}

# Loads into Docker the stick's images that are missing on the host
rcns_load_docker() {
    [ "\$RCNS_ENGINE" = docker ] || return 0
    while IFS=\$'\\t' read -r img tgz; do
        [ -n "\$img" ] || continue
        if ! docker image inspect "\$img" >/dev/null 2>&1; then
            echo "Loading \$img into Docker…"
            gunzip -c "\$R/containers/docker/\$tgz" | docker load
        fi
    done < "\$R/containers/images.tsv"
}
EOF

cat > "$R/conf/usb.config" <<'EOF'
// usb.config: stick-specific configuration (loaded with -c by run.sh and the interface)
// The stick is mounted into the containers through --containerBindPaths (added by
// run.sh and the interface), which nextflow.config turns into -B or -v.
def usb = System.getenv('RCNS_USB')

singularity.cacheDir = "${usb}/containers/sif"
apptainer.cacheDir   = "${usb}/containers/sif"

// Steps that fail on simulated BAMs (no SV, no heterozygous SNP).
// COMMENT OUT for real data, so that errors do not go unnoticed.
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
EOF

cat > "$R/run.sh" <<'EOF'
#!/usr/bin/env bash
# run.sh: runs Rapid-CNS2 from the stick
#   ./run.sh --check                      check the installation
#   ./run.sh BAM [ID] [nextflow options]  analyse one sample
set -euo pipefail
source "$(cd "$(dirname "$(readlink -f "$0")")" && pwd)/env.sh"

if [ "${1:-}" = --check ]; then
    ok=1
    chk() { if eval "$2" >/dev/null 2>&1; then echo "  ✔ $1"; else echo "  ✘ $1"; ok=0; fi; }
    echo "Stick: $R"
    echo "Container engine: $RCNS_ENGINE"
    chk "usable container engine"          "[ '$RCNS_ENGINE' != none ]"
    chk "Java"                             "java -version"
    chk "Nextflow (offline)"               "nextflow -version"
    chk "pipeline"                         "[ -f '$RCNS_PIPELINE' ]"
    chk "reference + index"                "[ -f '$RCNS_REF' ] && [ -f '$RCNS_REF.fai' ]"
    chk "ANNOVAR (table_annovar.pl)"       "[ -f '$RCNS_ANNOVARPATH/table_annovar.pl' ]"
    chk "ANNOVAR databases"                "[ -f '$RCNS_ANNOVARDB/hg38_refGene.txt' ]"
    chk "AnnotSV"                          "[ -d '$RCNS_ANNOTSVANNOT/Annotations_Human' ]"
    chk "work directory writable"          "mkdir -p '$RCNS_RUNDIR'"
    if [ -d "$RCNS_RUNDIR" ]; then
        echo "Work directory: $RCNS_RUNDIR ($(df -h --output=avail "$RCNS_RUNDIR" | tail -1 | tr -d ' ') free; allow ~150
GB/sample)"
    fi
```

```
    echo "Resources: $RCNS_MAXCPUS CPUs, $RCNS_MAXMEMORY GB"
    echo "Samples found: $(find "$RCNS_COHORT" -maxdepth 2 -name '*.bam' 2>/dev/null | wc -l)"
    [ "$ok" = 1 ] && echo "Everything is in place." || { echo "Some items are missing."; exit 1; }
    exit 0
fi

[ $# -ge 1 ] || { sed -n '2,5p' "$0"; exit 1; }
BAM=$(readlink -f "$1"); shift
ID=$(basename "$BAM" .bam)
if [ $# -ge 1 ] && [ "${1#-}" = "$1" ]; then ID=$1; shift; fi
[ -f "$BAM" ] || { echo "BAM not found: $BAM" >&2; exit 1; }

rcns_check_engine
rcns_load_docker
mkdir -p "$RCNS_RUNDIR"; cd "$RCNS_RUNDIR"
OUT=${RCNS_OUTDIR//\{id\}/$ID}

exec nextflow run "$RCNS_PIPELINE" -ansi-log false \
    --input "$BAM" --id "$ID" --outDir "$OUT" \
    --ref "$RCNS_REF" \
    --annovarPath "$RCNS_ANNOVARPATH" --annovarDB "$RCNS_ANNOVARDB" \
    --annotsvAnnot "$RCNS_ANNOTSVANNOT" \
    --containerBindPaths "$RCNS_BINDPATHS" \
    --minimumMgmtCov 10 --mnpFlex true --runHumanVariation "$RCNS_HUMANVAR" \
    --savanaCnaArgs '--overrule_cellularity 0.8' \
    --maxCpus "$RCNS_MAXCPUS" --maxMemory "$RCNS_MAXMEMORY" \
    -profile "$RCNS_PROFILE" -c "$RCNS_CONFIG" -resume "$@"
EOF

cat > "$R/run_gui.sh" <<'EOF'
#!/usr/bin/env bash
# run_gui.sh: opens the graphical interface, preconfigured for the stick
set -euo pipefail
source "$(cd "$(dirname "$(readlink -f "$0")")" && pwd)/env.sh"
[ -f "$R/gui/rapidcns2_gui.py" ] || { echo "graphical interface missing from the stick: use ./run.sh" >&2; exit 1; }
python3 -c 'import tkinter' 2>/dev/null || { echo "tkinter missing: sudo apt install python3-tk" >&2; exit 1; }
rcns_check_engine
rcns_load_docker
mkdir -p "$RCNS_RUNDIR"; cd "$RCNS_RUNDIR"
exec python3 "$R/gui/rapidcns2_gui.py"
EOF
chmod +x "$R/run.sh" "$R/run_gui.sh"

cat > "$R/README.txt" <<EOF
Rapid-CNS2 on a USB stick
=========================
Built on $(date '+%Y-%m-%d') on $(hostname)
Pipeline: Rapid-CNS2_nf, commit $PIPE_COMMIT
Nextflow: ${NF_VER:-?}    Containers: $ENGINE
ANNOVAR included: $([ "$WITH_ANNOVAR" = 1 ] && echo yes || echo no)

Requirements on the host computer
  - Linux x86_64
  - Apptainer/Singularity (recommended) or Docker
  - python3-tk for the graphical interface
  - ~150 GB free for the work directory (default \$HOME/rapidcns2_work,
    can be changed: export RCNS_WORKDIR=/path/to/disk)

Usage
  ./run.sh --check                    check the installation on this host
  ./run_gui.sh                        graphical interface
  ./run.sh samples/X/X.bam X          command line
Results: results/<id>/   Report: results/<id>/report/<id>_Rapid-CNS2_report_full.html

Notes
  - wf-human-variation is disabled: it downloads its own workflow and
    containers, which is impossible offline (with network: export RCNS_HUMANVAR=true).
  - Offline, automatic upload to MNP-Flex is not possible;
    the file mnpflex/<id>.MNPFlex.input.bed can be uploaded by hand.
  - conf/usb.config ignores AnnotSV and SAVANA failures (simulated BAMs):
    comment these lines out for real data.
  - ANNOVAR is free for academic and non-profit use and may not be
```

```
    redistributed: do not hand a stick containing it to a third party.
EOF

sync
say "Done"
echo "Stick ready in $R ($(du -sh "$R" | cut -f1))."
echo "Check: $R/run.sh --check"
```